

Week_4_Exercises

Exercise A: Segmentation

With the skills from Input: Segmentation you can now implement the segmentation algorithm described in Laube and Purves (2011) to either your own movement data or to a different wild boar using different sampling intervals.

```
library(tidyverse)
library(sf)
library(tmap)

# load data into R. I use the one from the whales to see if it works.
whales <- read_delim("Movements of Australia's east coast humpback whales.csv")

unique(whales$`individual-local-identifier`)
```

```
[1] 53348 53359 53376 53383 64235 64238 88717 88718 88723 88725 88728 88729
[13] 88730 88732 88733 88734 88735 88736 88737 88738 88741 88742 88743 88744
[25] 88745 88746 88747 88748 88750 88751 88752 88753 88754 88755 88756 96385
[37] 96386 96390 96395 96398 96401 96403 96404 96412 98100 98109 98114 98129
[49] 98138 98139
```

```
# I choose the first whale (53348) for this exercise

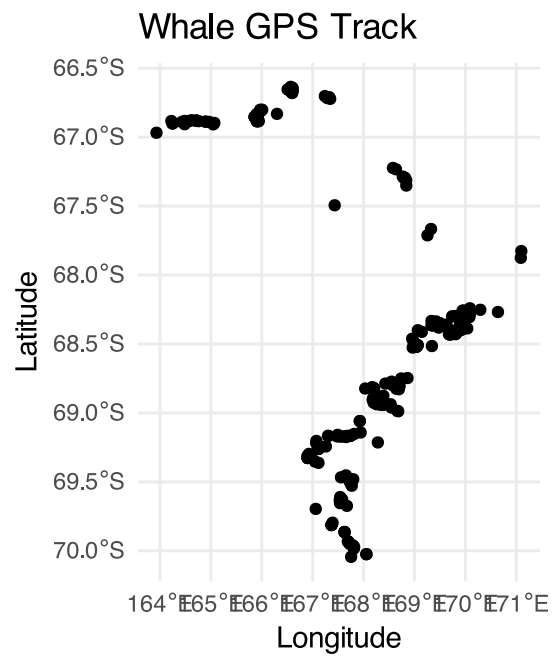
sample_whale <- whales |>
  st_as_sf(coords = c("location-long", "location-lat"), crs = 4326, remove =
FALSE) |>
  filter(
    `individual-local-identifier` == "53348",
    timestamp >= "2008-10-23",
    timestamp < "2011-07-26"
  )

#delete rows that are double
sample_whale <- sample_whale |>
  filter(!is.na(`argos:lat`))
```

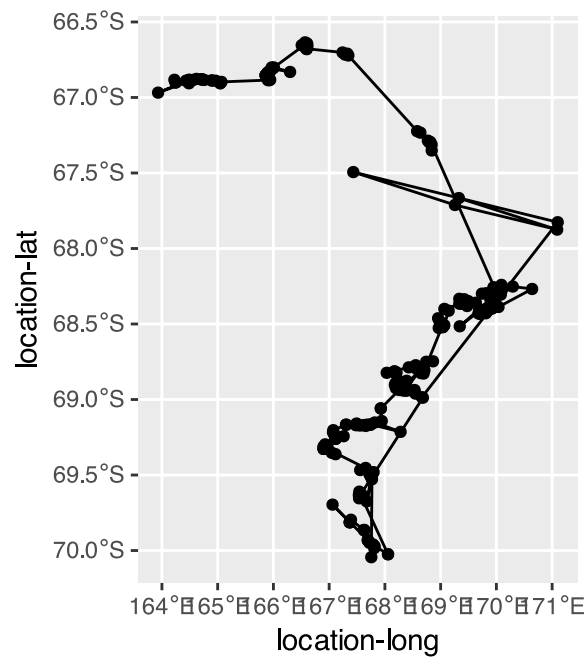
```
Error in `stopifnot()` :
i In argument: `!is.na(`argos:lat`))`.
```

Caused by error:
! object 'argos:lat' not found

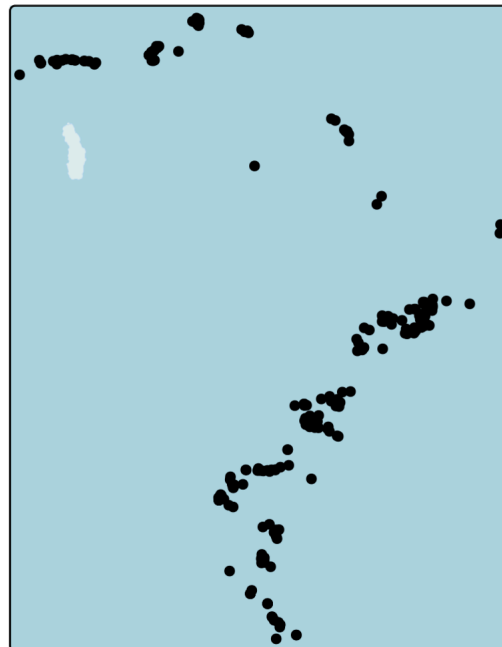
```
# show with ggplot
ggplot(sample_whale) +
  geom_sf() +
  labs(
    x = "Longitude",
    y = "Latitude",
    title = "Whale GPS Track"
  ) +
  theme_minimal()
```



```
ggplot(data = sample_whale) +
  geom_sf() +
  geom_path(aes(`location-long`, `location-lat`))
```



```
# with tmap
tm_shape(sample_whale) +
  tm_basemap("OpenStreetMap") +
  tm_dots()
```



specify a temporal window

sampling window is 30 seconds from 00:00-04:00 UTC and 12:00-16:00 UTC

This means, that I have 2 periods of 4h where sampling takes place. Therefore, I need to specify it!

Also I choose a temporal window of 120 seconds, thus 4 fixes.

```
# I create a function for distance
distance_by_element <- function(later, now) {
  as.numeric(
    st_distance(later, now, by_element = TRUE)
  )
}

sampling_v_whale <- sample_whale |>
  mutate(t_minus_120 = timestamp - 120,
         t_minus_60 = timestamp - 60,
         t_plus_60 = timestamp + 60,
         t_plus_120 = timestamp + 120)

sampling_v_whale <- sample_whale |>
  mutate(
    gap = as.numeric(difftime(timestamp, lag(timestamp), units = "secs")),
    segment = cumsum(ifelse(is.na(gap) | gap > 120, 1, 0))
  )

sampling_v_whale <- sampling_v_whale |>
  group_by(segment) |>
  mutate(
    nMinus2 = distance_by_element(lag(geometry, 4), geometry),
    nMinus1 = distance_by_element(lag(geometry, 2), geometry),
    nPlus1 = distance_by_element(geometry, lead(geometry, 2)),
    nPlus2 = distance_by_element(geometry, lead(geometry, 4))
  ) |>
  ungroup()
```

I receive lots of NA's. I believe the data needs to be cleaned properly before I can do any calculations. Therefore I use now the wildboar data again.

```
wildschwein <- read_delim("wildschwein_BE_2056.csv")

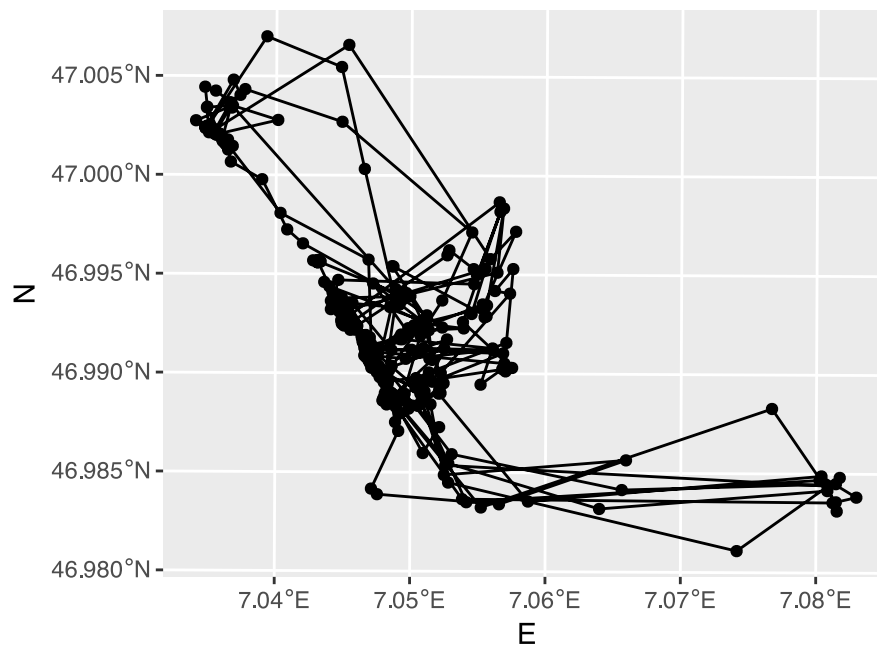
unique(wildschwein$TierName)
```

```
[1] "Sabi" "Rosa" "Ruth"
```

```
# convert to a spatial object
Rosa <- wildschwein |>
  st_as_sf(coords = c("E", "N"), crs = 2056, remove = FALSE) |>
  filter(
    TierName == "Rosa",
    DatetimeUTC >= "2014-11-07",
    DatetimeUTC < "2014-12-31"
  )

# visualise data

ggplot(data = Rosa) +
  geom_sf() +
  geom_path(aes(E, N)) # also connect dots with lines. This respects the order
# the data is in. geom_line would simply go from left to right.
```



Task 1: Calculate distances

Now, you can Step a): Specify a temporal window v and Step b): Measure the distance to every point within v , which you had used with `sabi`, on your own movement data or to a different wild boar using different sampling intervals.

```

distance_by_element <- function(later, now) {
  as.numeric(
    st_distance(later, now, by_element = TRUE)
  )
}

Rosa <- Rosa |>
  mutate(nMinus2 = distance_by_element(lag(geometry, 2), geometry)) |> # distance
to pos -30 minutes
  mutate(nMinus1 = distance_by_element(lag(geometry, 1), geometry)) |> # distance
to pos -15 minutes
  mutate(nPlus1 = distance_by_element(geometry, lead(geometry, 1))) |> # distance
to pos +15 minutes
  mutate(nPlus2 = distance_by_element(geometry, lead(geometry, 2))) # distance
to pos +30 minutes

# now calculate distances

Rosa <- Rosa |>
  rowwise() |>
  mutate(stepMean = mean(c(nMinus2, nMinus1, nPlus1, nPlus2), na.rm = FALSE))
|>
  ungroup() #

```

Task 2: Specify and apply threshold d

After calculating the Euclidean distances to positions within the temporal window v in task 1, you can explore these values (we stored them in the column `stepMean`) using summary statistics (histograms, boxplot, `summary()`): This way we can define a reasonable threshold value to differentiate between stops and moves. There is no “correct” way of doing this, specifying a threshold always depends on data as well as the question that needs to be answered. In this exercise, use the mean of all `stepMean` values.

Store the new information (boolean to differentiate between stops (TRUE) and moves (FALSE)) in a new column named `static`.

```

# set a threshold
threshold <- mean(Rosa$stepMean)

head(Rosa)

```

```

Simple feature collection with 6 features and 11 fields
Geometry type: POINT
Dimension:      XY

```

```

Bounding box: xmin: 2570375 ymin: 1204787 xmax: 2570389 ymax: 1204857
Projected CRS: CH1903+ / LV95
# A tibble: 6 × 12
  TierID TierName CollarID DatetimeUTC           E           N
  <chr>   <chr>      <dbl> <dtm>          <dbl>      <dbl>
1 016A   Rosa        13972 2014-11-07 07:45:44 2570385. 1204812.
2 016A   Rosa        13972 2014-11-07 08:00:10 2570389. 1204821.
3 016A   Rosa        13972 2014-11-07 08:15:18 2570375. 1204787.
4 016A   Rosa        13972 2014-11-07 08:30:23 2570387. 1204828.
5 016A   Rosa        13972 2014-11-07 08:46:08 2570387. 1204853.
6 016A   Rosa        13972 2014-11-07 09:00:11 2570387. 1204857.
# i 6 more variables: geometry <POINT [m]>, nMinus2 <dbl>, nMinus1 <dbl>,
#   nPlus1 <dbl>, nPlus2 <dbl>, stepMean <dbl>

```

```

# we define the threshold here as the mean.
Rosa <- Rosa |>
  mutate(static = stepMean < mean(stepMean, na.rm = TRUE))

Rosa_filter <- Rosa |>
  filter(!static) # these points are not static!

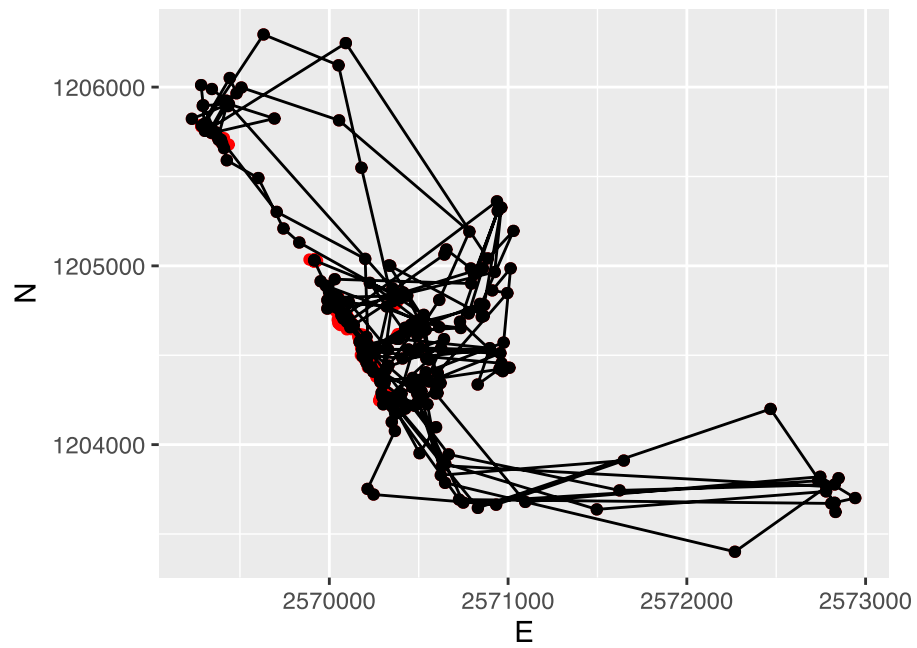
```

Task 3: Visualize segmented trajectories

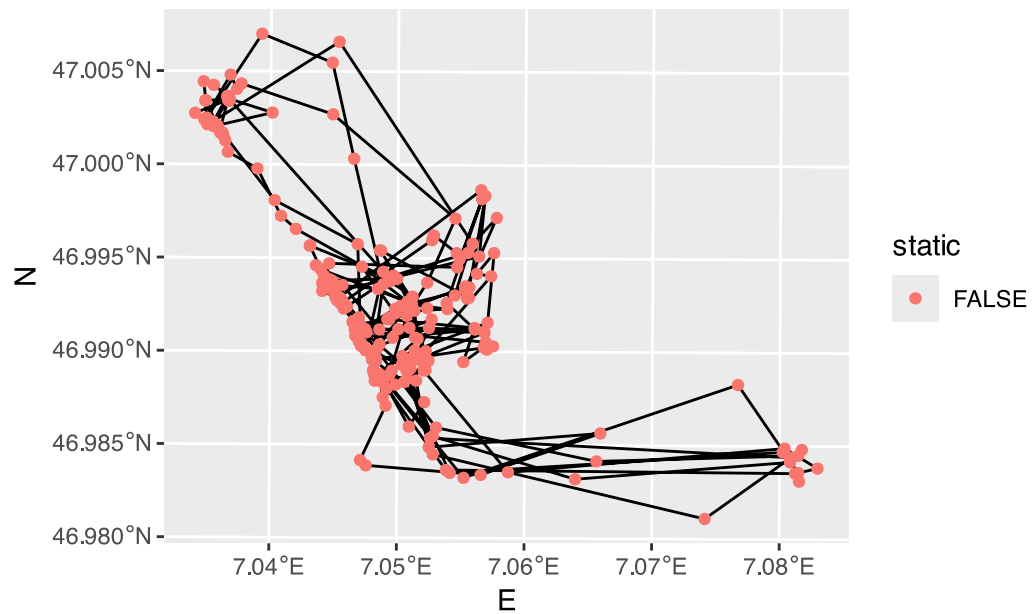
```

# shows static points in red
Rosa_filter |>
  ggplot(aes(E, N)) +
  geom_point(data = Rosa, col = "red") + # this adds the static points
  geom_path() +
  geom_point() +
  coord_fixed() +
  theme(legend.position = "bottom")

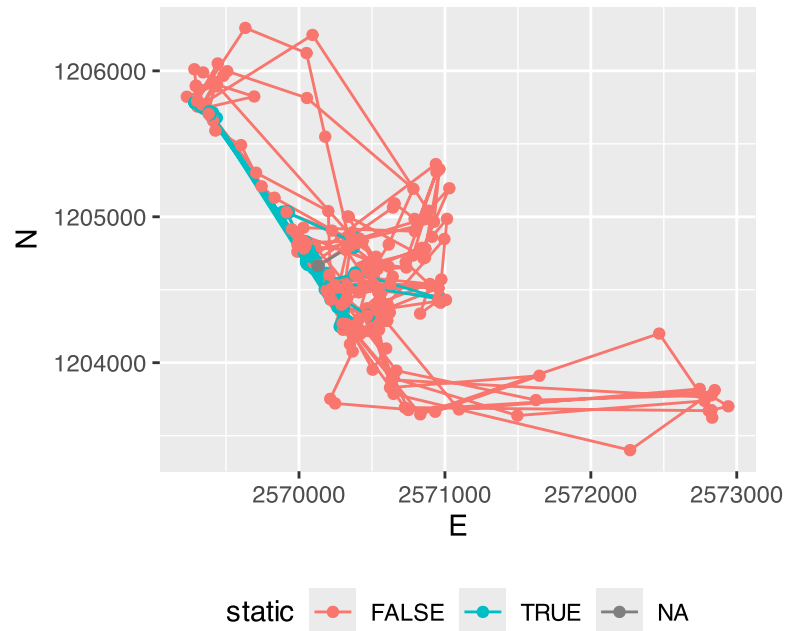
```



```
# show moving points
ggplot(Rosa_filter) +
  geom_path(aes(E, N)) +
  geom_sf(aes(color = static))
```




```
Rosa |>
  ggplot(aes(E, N, colour = static)) +
  geom_path() +
  geom_point() +
  coord_fixed() +
  theme(legend.position = "bottom")
```



Task 4: Segment-based analysis

In applying Laube and Purves (2011), we've come as far as step b) in Figure 16.1. In order to complete the last steps (c and d), we need a unique ID for each segment that we can use as a grouping variable. The following function does just that (it assigns unique IDs based on the column `static` which you created in Task 2). You will learn about functions next week. For now, just copy the following code chunk into your script and run it.

```
rle_id <- function(vec) {
  x <- rle(vec)$lengths
  as.factor(rep(seq_along(x), times = x))
}
```

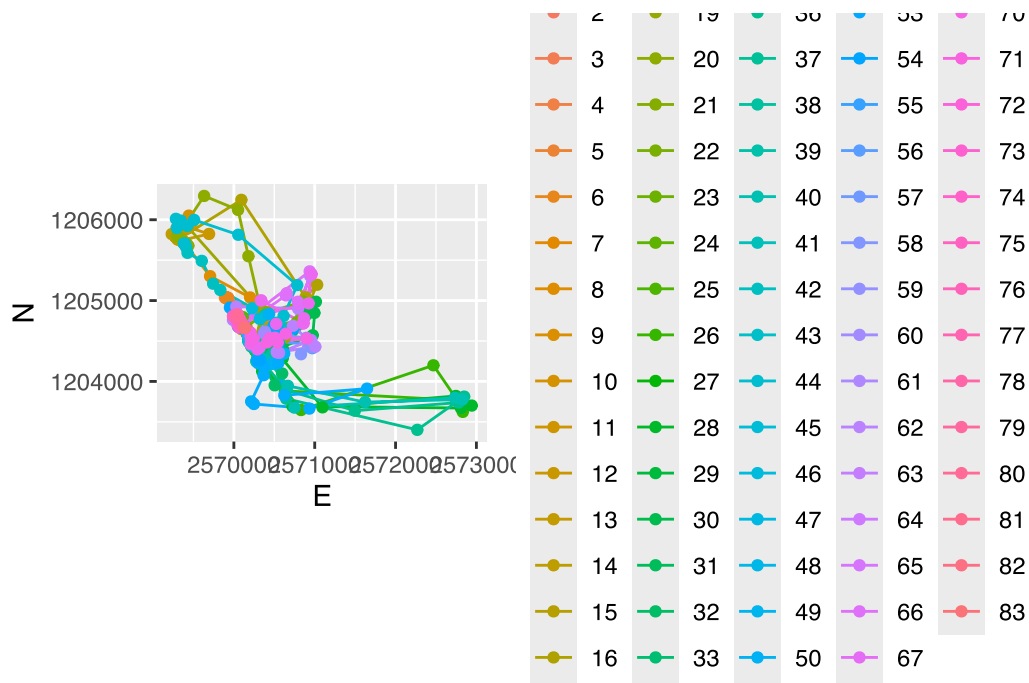
You can use the newly created function `rle_id` to assign unique IDs to subtrajectories (as shown below).

```
Rosa <- Rosa |>
  mutate(segment_id = rle_id(static))
```

Visualize the moving segments by colourizing them by segment_ID. Then use segment_ID as a grouping variable to determine the segments duration and remove short segments (e.g. segments with a duration < 5 Minutes)

```
# visualise

Rosa |>
  ggplot(aes(E, N, colour = segment_id)) +
  geom_path() +
  geom_point() +
  coord_fixed() +
  theme(legend.position = "right")
```



```
# group by segment ID and filter out duration more than 5mins.

Valid_segments <- Rosa |>
  group_by(segment_id) |>
  summarise(start = min(DatetimeUTC),
            end = max(DatetimeUTC),
            duration = as.numeric(difftime(end, start, units = "mins"))) |>
```

```

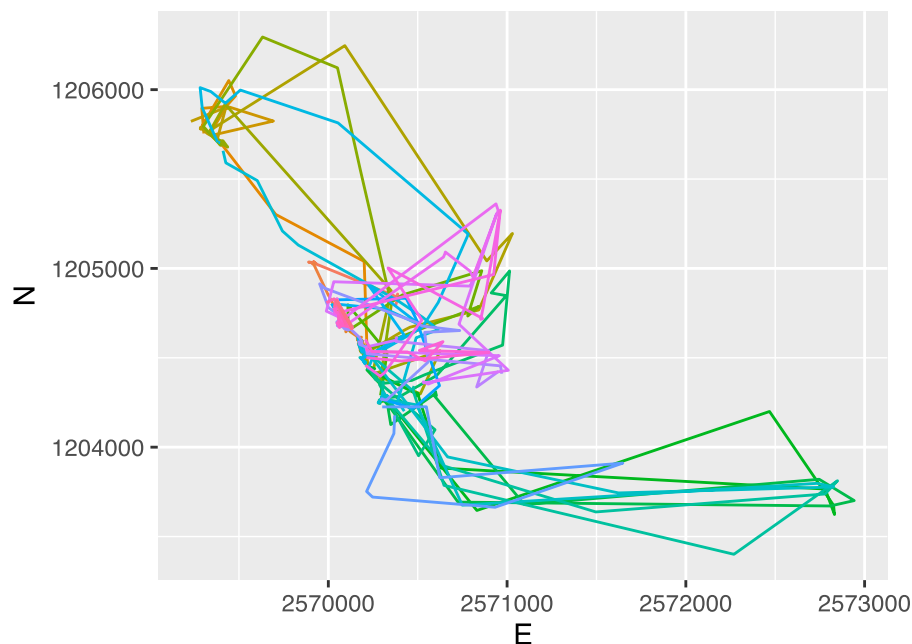
filter(duration >= 5)

Rosa <- Rosa |>
  filter(segment_id %in% Valid_segments$segment_id)

# now plot again:

Rosa |>
  ggplot(aes(E, N, colour = segment_id, group = segment_id)) +
    geom_path() +
    coord_fixed() +
    theme(legend.position = "")

```



Exercise B: Similarity

Task 1: Similarity measures

We will now calculate similarities between trajectories using a new dataset `pedestrian.csv` (available on moodle). Download and import this dataset as a data frame or tibble. It is a set of six different but similar trajectories from pedestrians walking on a path.

For this task, explore the trajectories first and get an idea on how the pedestrians moved.

```

pedestrian <- read_delim("pedestrian.csv")

```

```
pedestrian |>
  st_as_sf(coords = c("E", "N"), crs = 2056, remove = FALSE)
```

Simple feature collection with 289 features and 4 fields

Geometry type: POINT

Dimension: XY

Bounding box: xmin: 2569231 ymin: 1205132 xmax: 2571414 ymax: 1206556

Projected CRS: CH1903+ / LV95

A tibble: 289 × 5

	TrajID	E	N	DatetimeUTC	geometry
*	<dbl>	<dbl>	<dbl>	<dtm>	<POINT [m]>
1	1	2571414.	1205804.	2015-03-01 12:01:00	(2571414 1205804)
2	1	2571396.	1205791.	2015-03-01 12:02:00	(2571396 1205791)
3	1	2571373.	1205770.	2015-03-01 12:03:00	(2571373 1205770)
4	1	2571347.	1205753.	2015-03-01 12:04:00	(2571347 1205753)
5	1	2571336.	1205744.	2015-03-01 12:05:00	(2571336 1205744)
6	1	2571321.	1205732.	2015-03-01 12:06:00	(2571321 1205732)
7	1	2571304.	1205723.	2015-03-01 12:07:00	(2571304 1205723)
8	1	2571285.	1205706.	2015-03-01 12:08:00	(2571285 1205706)
9	1	2571253.	1205694.	2015-03-01 12:09:00	(2571253 1205694)
10	1	2571219.	1205674.	2015-03-01 12:10:00	(2571219 1205674)

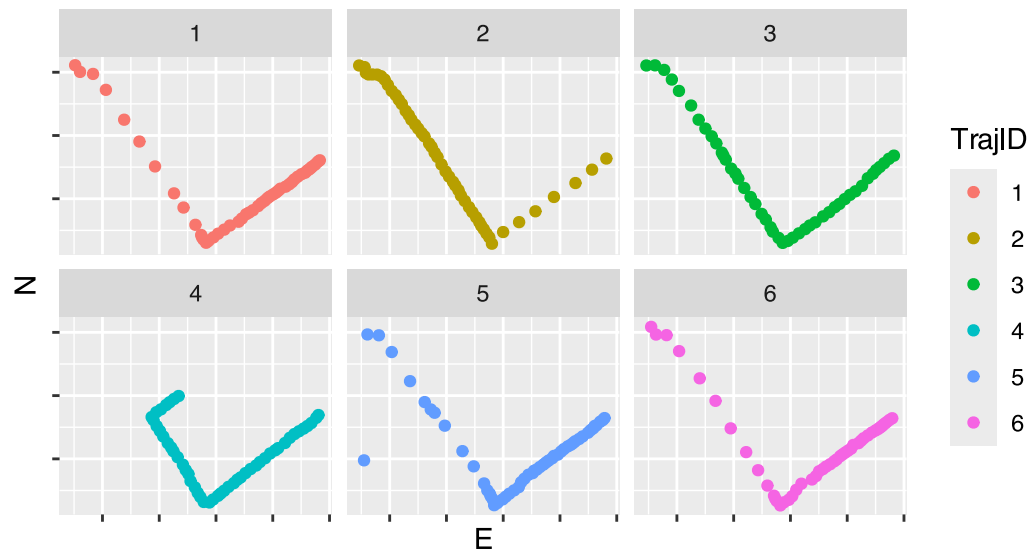
i 279 more rows

```
pedestrian$TrajID <- as.factor(pedestrian$TrajID)

ggplot(pedestrian, aes(E, N, colour = TrajID)) +
  geom_point() +
  facet_wrap(~TrajID) +
  theme(axis.text.x = element_blank(),
        axis.text.y = element_blank(),
  ) +
  labs(x = "E", y = "N",
       title = "Visual comparison of the 6 trajectories",
       subtitle = "Each subplot highlights a trajectory")
```

Visual comparison of the 6 trajectories

Each subplot highlights a trajectory



Task 2: Calculate similarity

```
library(SimilarityMeasures)
```

Familiarize yourself with this package by skimming through the function descriptions `help(package = "SimilarityMeasures")`. Now compare trajectory 1 to trajectories 2-6 using different similarity measures from the package. Your options are. DTW, EditDist, Frechet and LCSS.

All functions in the package need matrices as input, with one trajectory per matrix.

create matrices

```
t1 <- pedestrian |>
  filter(TrajID == 1) |>
  select(E, N) |>
  as.matrix()

t2 <- pedestrian |>
  filter(TrajID == 2) |>
  select(E, N) |>
  as.matrix()

t3 <- pedestrian |>
  filter(TrajID == 3) |>
  select(E, N) |>
```

```

as.matrix()

t4 <- pedestrian |>
  filter(TrajID == 4) |>
  select(E, N) |>
  as.matrix()

t5 <- pedestrian |>
  filter(TrajID == 5) |>
  select(E, N) |>
  as.matrix()

t6 <- pedestrian |>
  filter(TrajID == 6) |>
  select(E, N) |>
  as.matrix()

```

Edit Distance

```
EditDist(traj1 = t1, traj2 = t2, pointDistance = 20) #45
```

```
[1] 45
```

```
EditDist(traj1 = t1, traj2 = t3, pointDistance = 20) #47
```

```
[1] 47
```

```
EditDist(traj1 = t1, traj2 = t4, pointDistance = 20) #42
```

```
[1] 42
```

```
EditDist(traj1 = t1, traj2 = t5, pointDistance = 20) #28
```

```
[1] 28
```

```
EditDist(traj1 = t1, traj2 = t6, pointDistance = 20) #27
```

```
[1] 27
```

```

table_df <- data.frame(
  TrajID = c(2, 3, 4, 5, 6),
  edit_dist = c(45, 47, 42, 28, 27)
)

table_df$TrajID <- as.factor(table_df$TrajID)

p1 <- ggplot(table_df, aes(TrajID, edit_dist, fill = TrajID)) +
  geom_col() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(x = "Trajectory ID", y = "Edit distance")

```

DTW

```
DTW(traj1 = t1, traj2 = t2, pointSpacing=-1) # 3650.025
```

```
[1] 3650.025
```

```
DTW(traj1 = t1, traj2 = t3, pointSpacing=-1) # 50785.51
```

```
[1] 50785.51
```

```
DTW(traj1 = t1, traj2 = t4, pointSpacing=-1) # 5906.787
```

```
[1] 5906.787
```

```
DTW(traj1 = t1, traj2 = t5, pointSpacing=-1) # 2178.411
```

```
[1] 2178.411
```

```
DTW(traj1 = t1, traj2 = t6, pointSpacing=-1) # 1152.718
```

```
[1] 1152.718
```

```

table_df <- table_df |>
  mutate(dtw_all = c(3650.025, 50785.51, 5906.787, 2178.411, 1152.718))

```

```
p2 <- ggplot(table_df, aes(TrajID, dtw_all, fill = TrajID)) +
  geom_col() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(x = "Trajectory ID", y = "DTW")
```

Frechet distance

```
Frechet(traj1 = t1, traj2 = t2, testLeash=-1) #28.54075
```

```
28.54075
```

```
Frechet(traj1 = t1, traj2 = t3, testLeash=-1) #2307.844
```

```
E
2307.844
```

```
Frechet(traj1 = t1, traj2 = t4, testLeash=-1) #1069.229
```

```
1069.229
```

```
Frechet(traj1 = t1, traj2 = t5, testLeash=-1) #717.9816
```

```
717.9816
```

```
Frechet(traj1 = t1, traj2 = t6, testLeash=-1) #38.96272
```

```
38.96272
```

```
table_df <- table_df |>
  mutate(frechet_all = c(28.54075, 2307.844, 1069.229, 717.9816, 38.96272))
```



```
p3 <- ggplot(table_df, aes(TrajID, frechet_all, fill = TrajID)) +
  geom_col() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(x = "Trajectory ID", y = "Frechet Distance")
```

LCSS

LCSS takes very long to compute. The accuracy of the algorithm (`pointSpacing =` , `pointDistance =` and `errorMarg =`) can be varied to provide faster calculations. Please see Vlachos et al. (2002) for more information.

```
LCSS(traj1 = t1, traj2 = t2, pointSpacing=5, pointDistance=50,
     errorMarg=1, returnTrans=FALSE) #8
```

```
[1] 8
```

```
LCSS(traj1 = t1, traj2 = t3, pointSpacing=5, pointDistance=50,
     errorMarg=1, returnTrans=FALSE) #2
```

```
[1] 2
```

```
LCSS(traj1 = t1, traj2 = t4, pointSpacing=5, pointDistance=50,
     errorMarg=1, returnTrans=FALSE) #25
```

```
[1] 25
```

```
LCSS(traj1 = t1, traj2 = t5, pointSpacing=5, pointDistance=50,
     errorMarg=1, returnTrans=FALSE) #41
```

```
[1] 41
```

```
LCSS(traj1 = t1, traj2 = t6, pointSpacing=5, pointDistance=50,
     errorMarg=1, returnTrans=FALSE) #47
```

```
[1] 47
```

```

table_df <- table_df |>
  mutate(LCSS_all = c(8, 2, 25, 41, 47))

p4 <- ggplot(table_df, aes(TrajID, LCSS_all, fill = TrajID)) +
  geom_col() +
  theme_minimal() +
  theme(legend.position = "none") +
  labs(x = "Trajectory ID", y = "LCSS")

```

Before visualizing your results think about the following: Which two trajectories to you perceive to be most similar, which are most dissimilar?

-> most similar: TrajID 1 and 6

-> most dissimilar: TrajID 1 and 4 or 2

Now visualize the results from the computed similarity measures. Which measure reflects your own intuition the closest?

```

library(patchwork)

p1 + p2 + p3 + p4 +
  plot_layout(ncol = 2, nrow = 2) +
  plot_annotation(
    title = "Computed similarities using different measures between trajectory
1 and all other trajectories")

```

Computed similarities using different measures between trajectory

